В ESP-IDF v6 (FreeRTOS) у тебя есть несколько уровней инструментов — от “быстро посмотреть в лог” до полноценного runtime profiling и heap tracing.

Ниже — практический набор, который реально используют в проектах (включая BLE + IMU + UI).

🧠 1. Информация о задачах FreeRTOS (самое базовое)

✅ `vTaskList()` — список задач

```
char buffer[512];

void print_task_list(void)
{
    vTaskList(buffer);
    printf("\nTask Name    State   Prio   Stack   Num\n%s\n", buffer);
}
```

📌 Что показывает:

- имя задачи
- состояние (Running / Blocked / Ready)
- приоритет
- **остаток стека (stack high water mark)** ← ОЧЕНЬ важно
- номер задачи

⚠️ ВАЖНО

Чтобы работало:

```
CONFIG_FREERTOS_USE_TRACE_FACILITY=y
CONFIG_FREERTOS_USE_STATS_FORMATTING_FUNCTIONS=y
```

в `menuconfig`

🧠 2. Stack watermark (самый полезный инструмент)

Для конкретной задачи:

```
UBaseType_t free_stack = uxTaskGetStackHighWaterMark(NULL);
printf("Free stack: %d bytes\n", free_stack * 4);
```

👉 показывает:

- сколько стека осталось
- можно ловить stack overflow заранее

🧠 3. Runtime CPU load (нагрузка задач)

```
vTaskGetRunTimeStats(buffer);
printf("%s\n", buffer);
```

📌 Покажет:

- сколько CPU времени съела каждая задача
- % загрузки

⚠️ Включить нужно:

```
CONFIG_FREERTOS_GENERATE_RUN_TIME_STATS=y
CONFIG_FREERTOS_USE_STATS_FORMATTING_FUNCTIONS=y
```

🧠 4. Heap (память ESP32)

📌 Общая память

```
printf("Free heap: %lu\n", esp_get_free_heap_size());
```

📌 Самый полезный вариант:

```
printf("Min free heap: %lu\n", esp_get_minimum_free_heap_size());
```

👉 показывает худший момент за всё время

🧠 5. Детальная heap диагностика

```
heap_caps_print_heap_info(MALLOC_CAP_DEFAULT);
```

📌 Или конкретные типы памяти:

```
heap_caps_get_free_size(MALLOC_CAP_INTERNAL);
heap_caps_get_free_size(MALLOC_CAP_SPIRAM);
```

🧠 6. Heap tracing (очень мощно)

В ESP-IDF v6 есть:

```
heap_trace
```

включение:

```
heap_trace_init(STORE_AWARE);
heap_trace_start(HEAP_TRACE_ALL);
```

остановка:

```
heap_trace_stop();
heap_trace_dump();
```

👉 показывает:

- кто выделяет память
- где утечки
- стек аллокаций

🧠 7. Task-aware debugging в VSCode

Если используешь ESP-IDF extension:

👉 можно включить:

- FreeRTOS task view
- heap monitor
- stack usage

🧠 8. Watchdog + panic dump (очень важно)

В `menuconfig`:

```
Component config →
    ESP System Settings →
        Panic handler → GDB Stub or Print Backtrace
```

🔥 9. Самый полезный “debug task” (рекомендую)

Добавь отдельную задачу:

```
void debug_task(void *arg)
{
    char buf[512];
```

```
    while (1) {

        printf("\n=== TASK LIST ===\n");
        vTaskList(buf);
        printf("%s\n", buf);

        printf("Free heap: %lu\n", esp_get_free_heap_size());
        printf("Min heap:  %lu\n", esp_get_minimum_free_heap_size());

        printf("Stack BLE task: %d bytes\n",
            uxTaskGetStackHighWaterMark(NULL) * 4);

        vTaskDelay(pdMS_TO_TICKS(2000));
    }
}
```

🧠 10. Что важно именно для твоего проекта (BLE + IMU + Three.js)

У тебя критичны:

⚡ стек BLE task

```
uxTaskGetStackHighWaterMark(ble_task)
```

⚡ heap fragmentation

```
heap_caps_get_largest_free_block(MALLOC_CAP_DEFAULT)
```

⚡ CPU load (IMU filtering)

```
vTaskGetRunTimeStats()
```

🚀 Итог (коротко)

В ESP-IDF v6 у тебя есть 4 уровня:

🟢 Быстро

- `esp_get_free_heap_size()`

🟡 Нормально

- `vTaskList()`
- `uxTaskGetStackHighWaterMark()`

🟠 Продвинутый

- `vTaskGetRunTimeStats()`
- `heap_caps_print_heap_info()`

🔴 Профи

- heap tracing
- GDB + FreeRTOS aware debugging